

Samadhan Setu — Full Backend Technical Manual

API Architecture, Database Schemas, Duplicate Detection Algorithm & 10 Presentation Q&As;

1. Backend Infrastructure & System Architecture

The backend is constructed as a RESTful API service using Node.js and Express.js, utilizing Mongoose ODM for object mapping against MongoDB. It features strict schema validation, role-based JWT authorization, multi-file media processing via Multer, and incorporates an embedded in-memory MongoDB fallback engine (`mongodb-memory-server`) allowing zero-dependency automated startup and testing.

Component Layer	Technology & Package	Implementation Details & Responsibility
Server Framework	Node.js + Express 4.19	Modular REST API router, JSON middleware, static asset serving & error handlers.
Database / ODM	MongoDB + Mongoose 8.5	Document schema definitions, validation, compound indexes & query population.
DB Fallback Engine	MongoMemoryServer 10	Spawns embedded in-memory MongoDB daemon if local Mongo instance is offline.
Auth & Password Security	JWT + bcryptjs 2.4	Blowfish password salt hashing (factor 10) & 30-day JWT bearer tokens.
Media Processing	Multer 1.4	Multipart form-data processing, image MIME filtering & disk storage under <code>/uploads</code> .

2. Backend Directory Structure & File Map

- `server.js`: Main application entry point. Initializes Express middleware, routes, static file serving (`/uploads`), database connection with seed fallback, and error handling middleware.
- `config/db.js`: Database manager. Attempts connecting to standard MongoDB URI; if connection fails, spawns `MongoMemoryServer` dynamically to ensure zero-downtime execution.
- `models/Challenge.js`: Primary Mongoose model storing challenge title, description, location fields (state, district, locality, landmark, pincode), GIS coordinates (`latitude`, `longitude`), `regionType`, `affectedWho`, `baselineMetric`, category, severity, status, `statusHistory`, and `mergedInto` references.
- `models/User.js`: User authentication model storing name, email (unique index), hashed password, role (`citizen`, `institution`, `admin`), organization, partnerType, and interests.
- `models/Proposal.js`: Institutional resolution proposal model linking challenge ID and institution user ID with proposed technical solution, cost, timeline, and status.
- `models/ProgressUpdate.js`: Execution evidence model tracking progress updates posted by institutions during solution execution.
- `models/Vote.js`: Voting model with compound unique index `{ challenge: 1, user: 1 }` preventing duplicate upvotes at the database layer.
- `models/GroundCheck.js`: Post-resolution citizen verification model tracking verdicts (`Working`, `Improved`, `Unchanged`, `Worsened`).
- `controllers/challengeController.js`: Core business logic controller handling challenge creation, full-text searching, real-time duplicate scoring, report merging, voting, status updates, and ground-checks.
- `middleware/authMiddleware.js`: Security middleware providing `protect` token verification, `adminOnly` access enforcement, and `institutionOnly` authorization checks.

3. Core Backend Algorithms & Logic

- 1. Duplicate Search & Scoring Engine** (`findSimilarChallenges``): Tokenizes input title using regular expressions, strips common stop words ('the', 'in', 'near'), normalizes term variants ('streetlights' -> 'street light'), computes keyword overlap Jaccard scores against active challenges, adds geographic match bonuses (+2 points for matching locality, +1 point for matching district), and ranks top similar reports.
- 2. Atomic Report Merging Logic** (`mergeChallenges``): When an admin merges a duplicate report into a primary challenge, the duplicate's `mergedInto`` pointer is set to the primary challenge ID. The duplicate's votes (`survivor.votesCount += duplicate.votesCount``) and affected citizens (`survivor.affectedCount += duplicate.affectedCount``) are atomically transferred, and audit notes are logged in `statusHistory`` for both reports. Public query filters automatically hide merged reports (`mergedInto == null``).
- 3. 5-Stage Status Audit Machine**: Enforces status progression (`Pending` → `Open` → `Under Review` → `In Progress` → `Resolved``). Every status change appends a historic record into `statusHistory`` storing `status``, `changedBy``, `changedAt``, and `note``, guaranteeing 100% auditability.
- 4. Atomic Vote Deduplication**: Enforces compound unique index `{ challenge: 1, user: 1 }` on `Vote`` schema. Handles MongoDB duplicate key error code `11000`` gracefully to prevent double voting or race conditions.

4. Complete REST API Endpoint Reference

HTTP Method & Path	Access Level	Functionality & Output
POST /api/auth/register	Public	Registers user, hashes password & returns user profile + JWT token.
POST /api/auth/login	Public	Authenticates credentials & returns user profile + JWT token.
GET /api/challenges	Public	Fetches approved public challenges filtered by status/category/location.
POST /api/challenges	Private (Citizen)	Creates new challenge with coordinates & evidence photos in Pending state.
GET /api/challenges/similar	Public	Runs duplicate algorithm & returns ranked similar challenge candidates.
POST /api/challenges/:id/vote	Private	Atomically records user upvote & increments challenge votes count.
POST /api/challenges/:id/proposals	Private (Institution)	Submits technical solution proposal & advances status to Under Review.
PATCH /api/challenges/:id/status	Private (Admin)	Updates challenge lifecycle status & appends audit entry to history.
POST /api/challenges/:id/merge	Private (Admin)	Merges duplicate report into primary issue & transfers vote counts.

5. Top 10 Technical Presentation Q&As; (Backend)

Q1: How does the server handle MongoDB database connectivity in environments without local MongoDB installed?

Answer: In `config/db.js``, the server first attempts connecting to the standard MongoDB URI (`process.env.MONGO_URI`` or `mongodb://127.0.0.1:27017/samadhan-setu``). If local MongoDB is unavailable or fails to connect, the server catches the error and automatically spawns `MongoMemoryServer`` (an in-memory MongoDB server). This allows the entire backend to start up instantly with zero external database configuration, seeding demo accounts automatically.

Q2: How does the duplicate detection algorithm work?

Answer: The duplicate engine (`findSimilarChallenges` in challengeController.js`) receives query parameters (title` , category` , district` , locality`). It tokenizes and normalizes words using regular expressions and a custom STOP_WORDS` set. It then performs a candidate search, scoring candidate challenges based on title word overlap (Jaccard-like keyword matching) plus geographic match bonuses (+2 for matching locality, +1 for matching district). Candidates scoring highest are returned to the frontend.`

Q3: How is security and Role-Based Access Control (RBAC) enforced in the API routes?

Answer: Express route middleware (`authMiddleware.js``) enforces security in layers: `protect` decodes the JWT bearer token from the Authorization` header, verifies its signature using process.env.JWT_SECRET` , and attaches req.user` . Specialized middleware (adminOnly` , institutionOnly`) inspects req.user.role` . If a user with role citizen` tries to access /api/admin` or /api/challenges/:id/status` , the middleware blocks the request with HTTP 403 Forbidden` .`

Q4: How does the report merging mechanism preserve vote data and maintain data integrity?

Answer: When an admin calls `POST /api/challenges/:id/merge` with duplicateId` , the controller verifies both reports exist and that neither has already been merged. It sets duplicate.mergedInto = survivor._id` , transfers all votes (survivor.votesCount += duplicate.votesCount`), adds affected citizen counts (survivor.affectedCount += duplicate.affectedCount`), and logs audit notes in statusHistory` for both challenges. Future queries for public challenges filter out reports where mergedInto != null` .`

Q5: How are file uploads (photos & evidence) processed and stored?

Answer: We configure `multer` in routes/challengeRoutes.js` with diskStorage` , storing uploads in the backend /uploads` directory with unique timestamped filenames (Date.now() + '-' + originalName`). Multer file filters enforce image mime-types (image/jpeg` , image/png` , image/webp` ). Express serves these statically via app.use('/uploads` , express.static(path.join(__dirname, 'uploads'))` , returning relative file paths (e.g. /uploads/1724889600000-photo.jpg`) in JSON API responses.`

Q6: How do you prevent double-upvoting or race conditions during voting?

Answer: At the database level, the `Vote` collection enforces a compound unique index on { challenge: 1, user: 1 }` . When a vote request arrives, Vote.create()` is executed inside a try/catch` block. If a duplicate vote is attempted simultaneously, MongoDB rejects the write with error code 11000` (Duplicate Key Error), which our controller catches to return a clean 400 Bad Request` without corrupting vote counters.`

Q7: How are latitude and longitude geospatial coordinates stored and validated?

Answer: In `models/Challenge.js` , latitude` and longitude` are defined as Number` fields initialized to null` . During challenge creation in challengeController.js` , input values are validated to ensure they are numeric floats, sanitizing coordinates before storage. These coordinates are exposed in JSON payloads to feed the frontend Leaflet map engine.`

Q8: How is password storage secured?

Answer: Passwords are never stored in plaintext. In `seed.js` and authController.js` , passwords pass through bcryptjs.genSalt(10)` and bcryptjs.hash()` . During login, bcryptjs.compare()` checks submitted plaintext passwords against the stored hash without ever exposing raw password strings.`

Q9: What is the structure of the 5-stage status audit trail (`statusHistory`)?`

Answer: `statusHistory` is an embedded schema array inside Challenge.js` containing objects with fields { status, changedBy, changedAt, note }` . Whenever an admin or automated lifecycle event triggers a status update (e.g. Pending` -> Open`), a new audit object is appended to the array, ensuring complete historical traceability of who changed status and why.`

Q10: How does the server handle centralized error logging and unexpected runtime failures?

Answer: `server.js` registers a centralized error handling middleware (err, req, res, next` ) at the bottom of the middleware chain. Any error thrown in asynchronous controller handlers or passed via next(err)` is logged to standard error (console.error` ), returning a standardized JSON error object { message, error }` with appropriate HTTP status codes (400, 401, 403, 404, 500).`